

AbrenCare — Authentication API Frontend Integration Guide

Production API Contract | Version 2.0 | August 2026

Production frontend integration reference for the refined AbrenCare authentication API. This version supersedes the previous authentication guide.

1. Purpose

This guide is the frontend integration contract for the refined authentication views and application URLs. It replaces the previous authentication guide and reflects the current endpoint behavior.

Unless otherwise stated, authenticated endpoints require a valid JWT access token in the Authorization header.

2. API Base URL

Configure the backend base URL through the frontend environment configuration. The exact variable name depends on the frontend framework.

Example: NEXT_PUBLIC_API_URL=https://api.example.com/api

Example: VITE_API_URL=https://api.example.com/api

Append the endpoint path shown in this document to the configured base URL.

3. Authentication Header

Authenticated requests must send the JWT access token using the Bearer scheme.

Authorization: Bearer <access_token>

Do not place access or refresh tokens in URLs.

4. Endpoint Summary

Method	Endpoint	Authentication	Purpose
POST	/auth/register/	Public	Register a user
POST	/auth/login/	Public	JWT login
POST	/auth/logout/	JWT	Blacklist the supplied refresh token
GET	/auth/profile/	JWT	Get current user
PUT	/auth/profile/	JWT	Update current user
POST	/auth/account/deactivate/	JWT	Deactivate current account
POST	/auth/password/change/	JWT	Change password
POST	/auth/password/reset/request/	Public	Request reset OTP
POST	/auth/password/reset/verify/	Public	Verify OTP and obtain reset_token
POST	/auth/password/reset/confirm/	Reset token	Set new password
GET	/admin/users/	Admin	List users
POST	/auth/token/refresh/	Refresh token	Refresh access token (project-level route)

5. Registration

Endpoint: POST /auth/register/

Authentication: Public

Request example:

```
{
  "email": "john@example.com",
  "password": "StrongPassword123!",
  "first_name": "John",
  "last_name": "Doe"
}
```

Successful response: 201 Created

```
{
  "message": "User registered successfully.",
  "user": { "...": "UserSerializer output" }
}
```

The exact user fields are determined by `UserCreateSerializer` and `UserSerializer`.

6. Login

Endpoint: `POST /auth/login/`

Authentication: Public

Request example:

```
{
  "email": "john@example.com",
  "password": "StrongPassword123!"
}
```

The endpoint uses `CustomTokenObtainPairSerializer`. The exact response fields are determined by that serializer and may include access and refresh tokens plus user information.

Never store or log the user's password.

7. Token Refresh

Endpoint: `POST /auth/token/refresh/`

Authentication: Refresh token

Request:

```
{
  "refresh": "<refresh_token>"
}
```

Expected SimpleJWT response:

```
{
  "access": "<new_access_token>"
}
```

Register this endpoint at the project level if it is not already registered. The app-level `urls.py` supplied for this guide does not define the refresh route.

8. Current User Profile

`GET /auth/profile/` — Authentication required. Returns the current authenticated user.

`PUT /auth/profile/` — Authentication required. Partial updates are supported.

Example PUT body:

```
{
  "first_name": "John",
  "last_name": "Smith"
}
```

Both operations use the current authenticated user; the endpoint does not accept a user ID.

9. Account Deactivation

Endpoint: `POST /auth/account/deactivate/`

Authentication: JWT required

The endpoint deactivates the current account rather than permanently deleting it. Superuser accounts cannot be deactivated through this endpoint.

Successful response:

```
{
  "message": "Your account has been deactivated successfully."
}
```

After success, clear the frontend authentication state and redirect to the login page.

10. Logout

Endpoint: POST /auth/logout/

Authentication: JWT required

Request:

```
{
  "refresh": "<refresh_token>"
}
```

The backend validates that the supplied refresh token belongs to the currently authenticated user, then blacklists it.

Successful response:

```
{
  "message": "Logged out successfully."
}
```

After logout, clear the access token, refresh token, user state, and authenticated application state.

11. Change Password

Endpoint: POST /auth/password/change/

Authentication: JWT required

Request:

```
{
  "current_password": "OldPassword123!",
  "new_password": "NewStrongPassword456!"
}
```

The backend verifies the current password, applies Django password validation, checks password history, changes the password atomically, stores password history, and revokes outstanding refresh tokens.

Successful response:

```
{
  "message": "Password changed successfully. Please sign in again on other devices.",
  "timestamp": "2026-08-24T09:30:00+00:00"
}
```

The frontend should clear/reconcile its authentication state according to the token revocation policy and require authentication again when the current refresh token is no longer valid.

12. Password Reset Overview

The password reset flow has three API stages:

1. Request a reset code.
2. Verify the six-digit code and receive a temporary reset_token.
3. Submit the reset_token with the new password.

The OTP is not submitted again during the final confirmation request.

13. Request Reset Code

Endpoint: POST /auth/password/reset/request/

Authentication: Public

Request:

```
{
  "email": "john@example.com"
}
```

Successful response:

```
{
  "message": "If the email exists, a reset code has been sent."
}
```

The response is intentionally generic to reduce account enumeration. The frontend should display the same generic result regardless of whether an account exists.

The backend applies request throttling and an email cooldown. The reset email is dispatched asynchronously.

14. Verify Reset Code

Endpoint: POST /auth/password/reset/verify/

Authentication: Public

Request:

```
{
  "email": "john@example.com",
  "code": "123456"
}
```

Successful response:

```
{
  "message": "Code verified successfully.",
  "reset_token": "<temporary_reset_token>"
}
```

The frontend must retain the reset_token only for the reset flow and must not expose it in URLs or logs. The OTP has a limited lifetime and failed verification attempts are throttled/limited by the password-reset service.

15. Confirm Password Reset

Endpoint: POST /auth/password/reset/confirm/

Authentication: Public, using the temporary reset token

Request:

```
{
  "reset_token": "<temporary_reset_token>",
  "new_password": "NewStrongPassword123!"
}
```

Successful response:

```
{
  "message": "Password reset successfully. Please sign in with your new password."
}
```

The backend validates the reset token, validates the new password, changes the password, revokes all outstanding refresh tokens, and sends a confirmation email asynchronously.

After success, discard the reset token and redirect the user to login.

16. Admin User Listing

Endpoint: GET /admin/users/

Authentication: Admin user required

The endpoint returns the user list through `UserSerializer`. Backend permissions are authoritative; frontend admin-route checks are only a user-experience measure.

17. Authentication State

Recommended frontend states are: loading, authenticated, and unauthenticated.

Example:

```
{
  "status": "authenticated",
  "user": { "...": "current user" },
  "accessToken": "...",
  "refreshToken": "..."
}
```

Do not store passwords in frontend state or storage.

18. Startup Authentication Flow

Application loads → inspect authentication state/tokens → use the access token for authenticated API calls
→ call `GET /auth/profile/` to hydrate the current user.

If an authenticated request returns 401 because the access token expired, perform one refresh operation, store the new access token, and retry the original request once.

If refresh fails, clear authentication state and redirect to `/login`.

Prevent multiple concurrent 401 responses from triggering multiple refresh requests; use a single-flight refresh mechanism in the API client.

19. Recommended 401 Refresh Behavior

Centralize authentication handling in the API client/interceptor rather than implementing refresh logic inside individual screens.

Recommended flow:

Request → 401 → refresh access token → update token storage/state → retry original request once.

If the refresh endpoint fails, reject the original request, clear authentication state, and redirect to login.

Do not retry indefinitely and do not refresh in response to the refresh endpoint itself.

20. Frontend Route Protection

Public frontend routes may include `/login`, `/register`, `/forgot-password`, `/verify-reset-code`, and `/reset-password`.

Authenticated frontend routes may include `/dashboard`, `/profile`, `/settings`, and `/change-password`.

Admin frontend routes may include `/admin` and `/admin/users`.

Frontend route protection improves user experience; Django permissions remain the final security authority.

21. HTTP Status Codes

200 — successful operation.

201 — resource created, such as successful registration.

400 — validation or client error; display an appropriate normalized error.

401 — access token invalid/expired; attempt the centralized refresh flow.

403 — authenticated user lacks permission or account access is denied.

404 — resource not found.

500 — server error; show a generic user-facing error and avoid exposing internal details.

22. Error Handling

Backend errors may be general or field-based.

```
{
  "error": "Current password is incorrect."
}

{
  "email": ["Enter a valid email address."]
}

{
  "password": ["This password is too short."]
}
```

Normalize these shapes in the API client or form layer so components can display consistent messages.

23. Frontend Security Rules

Never store or transmit passwords except in the intended HTTPS request body.

Never place JWTs or reset tokens in URLs.

Never log access tokens, refresh tokens, reset tokens, OTP codes, or passwords.

Always use Authorization: Bearer <access_token> for authenticated API requests.

Clear authentication state on logout, failed refresh, and account deactivation.

Do not treat frontend role checks as security controls.

Use HTTPS in production.

Centralize token refresh and avoid infinite retry loops.

Treat reset_token as a short-lived secret and discard it immediately after password reset.

24. Complete Authentication Flows

Login: POST /auth/login/ → receive access/refresh → store according to the application's token strategy → GET /auth/profile/ → enter authenticated application.

Password reset: POST /auth/password/reset/request/ → receive OTP by email → POST /auth/password/reset/verify/ → receive reset_token → POST /auth/password/reset/confirm/ → password changed → redirect to login.

Password change: POST /auth/password/change/ → password changed and refresh tokens revoked → require/reconcile authentication again.

Logout: POST /auth/logout/ → refresh token blacklisted → clear local authentication state.

Deactivation: POST /auth/account/deactivate/ → account disabled and refresh tokens revoked → clear local authentication state.

25. Quick Reference

Method	Endpoint	Authentication	Purpose
POST	/auth/register/	Public	Register a user
POST	/auth/login/	Public	JWT login
POST	/auth/logout/	JWT	Blacklist the supplied refresh token
GET	/auth/profile/	JWT	Get current user
PUT	/auth/profile/	JWT	Update current user
POST	/auth/account/deactivate/	JWT	Deactivate current account
POST	/auth/password/change/	JWT	Change password
POST	/auth/password/reset/request/	Public	Request reset OTP
POST	/auth/password/reset/verify/	Public	Verify OTP and obtain reset_token
POST	/auth/password/reset/confirm/	Reset token	Set new password
GET	/admin/users/	Admin	List users

Method	Endpoint	Authentication	Purpose
POST	/auth/token/refresh/	Refresh token	Refresh access token (project-level route)

26. Implementation Notes

This document is based on the refined `views.py` and `app-level urls.py` supplied for the integration. Serializer output remains serializer-dependent.

The refresh endpoint `/auth/token/refresh/` is documented because the authentication architecture uses SimpleJWT refresh tokens, but it must be registered in the project-level URL configuration if it is not already present.

If the backend implementation changes, update this guide and the frontend API client together.